



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

Dreamer: Learning by Latent Imagination

Tommaso Piscitelli

Università degli Studi di Trieste

 Corso: Reinforcement Learning

 Anno Accademico 2025/26



Motivation and aims

- Intelligent agents can achieve goals in complex environments even though they never encounter the exact same situation. This ability requires **building representations** of the world from past experience that enable generalization to novel situations.
- There is **extensive literature** on learning a dynamics model and using this model to train a policy, but these works have demonstrated promising results on control tasks where the states are known and well defined and the observation is relatively low dimensional.
- Here we are interested in modelling dynamics observed from **high dimensional visual data** where our input is a sequence of raw pixel frames.
- In this project, I use the World Model architecture, already trained, proposed in the paper World Models, and combined it with an **actor critic algorithm**, that learn long horizon behaviours from raw frames purely on latent imagination

Paper contributions

World Models

David Ha¹ Jürgen Schmidhuber^{2,3}

¹Google Brain ²NNAISENSE ³Swiss AI Lab, IDSIA (USI & SUPSI)

Learning Latent Dynamics for Planning from Pixels

Danijar Hafner^{1,2} Timothy Lillicrap³ Ian Fischer⁴ Ruben Villegas^{1,5}
David Ha¹ Honglak Lee¹ James Davidson¹

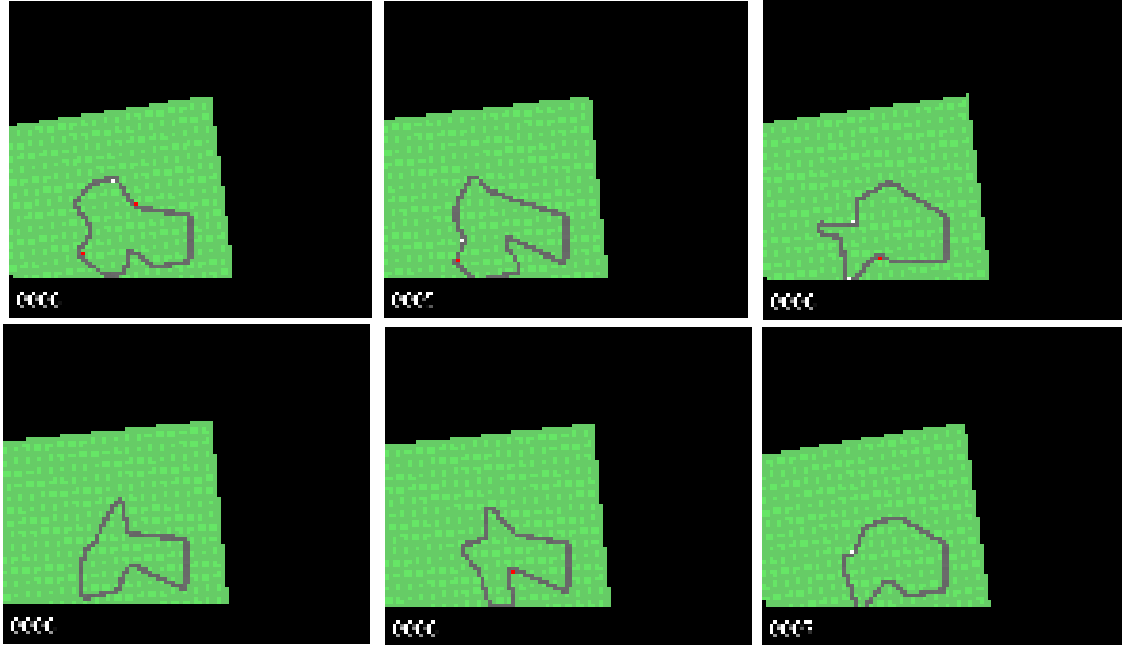
¹Google Brain ²University of Toronto ³DeepMind ⁴Google Research ⁵University of Michigan

DREAM TO CONTROL: LEARNING BEHAVIORS BY LATENT IMAGINATION

Danijar Hafner *	Timothy Lillicrap	Jimmy Ba	Mohammad Norouzi
University of Toronto	DeepMind	University of Toronto	Google Brain
Google Brain			

- Present a large neural network that learn a model of the agent's world in an unsupervised manner
- Using features extracting from the world model as input to an agent, train separately a compact and simple policy
- Demonstrated the possibility of training an agent entirely inside the simulated latent space world model
- Design a different latent dynamics model with both deterministic and stochastic components
- Generalize the standard variational bound to include multi step prediction
- Train jointly world model and behaviors and solve variety of tasks by efficiently planning in its world model
- Use the same world model and training procedure of previous paper and combined it with an actor critic algorithm (Dreamer)

CarRacing enviroment



Source:

Episodes generated from the OpenAI Gym / Gymnasium CarRacing environment. Each episode is saved as a .npz file containing observations and actions collected during environment rollouts.

Problem setup:

Transition function: $s_t \sim p(s_t \mid s_{t-1}, a_{t-1})$

Observation function: $o_t \sim p(o_t \mid s_t)$

Reward function: $r_t \sim p(r_t \mid s_t)$

Policy: $a_t \sim p(a_t \mid o_{\leq t}, a_{< t})$

Observations

Raw RGB (64×64×3) frames.

Actions

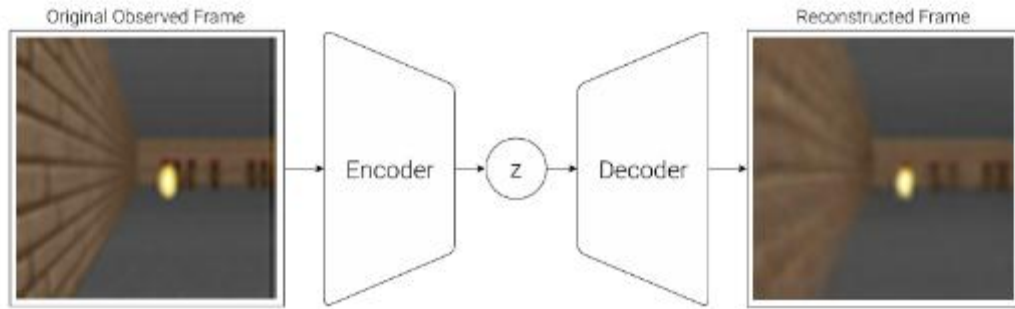
Continuous action vector:
 $a_t = [\text{steering}, \text{gas}, \text{brake}]$

Reward

The environment rewards the car for visiting new track tiles.

steering $\in [-1, 1]$
gas $\in [0, 1]$
brake $\in [0, 1]$

World model



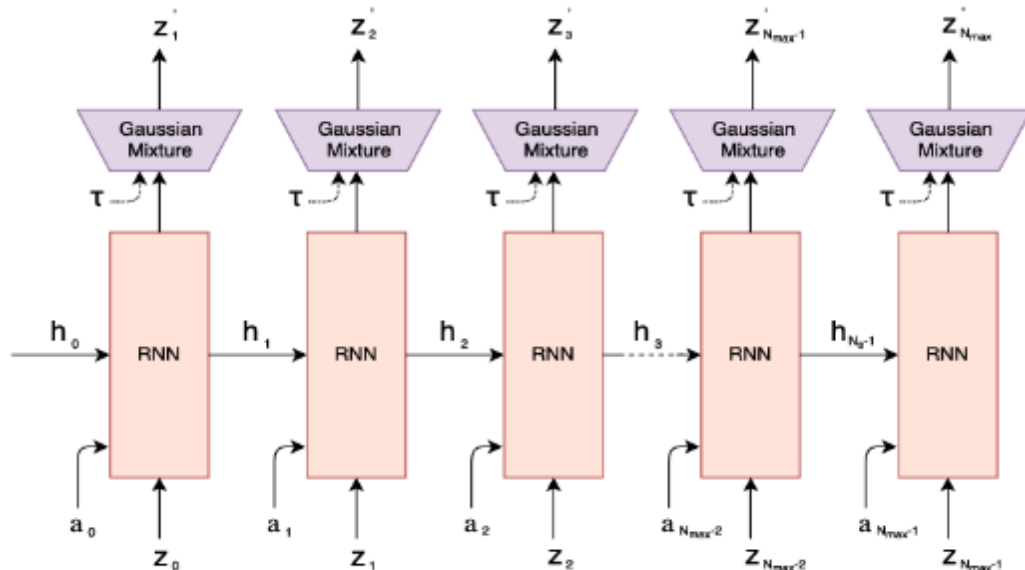
$$s_t = (z_t, h_t)$$

Components:

Representation $p_{\theta}(s_t \mid s_{t-1}, a_{t-1}, o_t)$

Transition $q_{\theta}(s_t \mid s_{t-1}, a_{t-1})$

Reward $q_{\theta}(r_t \mid s_t)$



This is **not strictly necessary** for the CarRacing environment. However, as discussed in the World Models paper, in more sophisticated environments the world model should also learn to predict the reward at the next time step. A controller can then be trained inside the world model to **maximize the expected reward** and subsequently used to **collect new data** from the real environment. This iterative process allows the representation model to focus on **task-relevant regions** of the observations.

Dreamer

Algorithm 1: Dreamer

```

Initialize dataset  $\mathcal{D}$  with  $S$  random seed episodes.
Initialize neural network parameters  $\theta, \phi, \psi$  randomly.
while not converged do
  for update step  $c = 1..C$  do
    // Dynamics learning
    Draw  $B$  data sequences  $\{(a_t, o_t, r_t)\}_{t=k}^{k+L} \sim \mathcal{D}$ .
    Compute model states  $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$ .
    Update  $\theta$  using representation learning.

    // Behavior learning
    Imagine trajectories  $\{(s_\tau, a_\tau)\}_{\tau=t}^{t+H}$  from each  $s_t$ .
    Predict rewards  $E(q_\theta(r_\tau | s_\tau))$  and values  $v_\psi(s_\tau)$ .
    Compute value estimates  $V_\lambda(s_\tau)$  via Equation 6.
    Update  $\phi \leftarrow \phi + \alpha \nabla_\phi \sum_{\tau=t}^{t+H} V_\lambda(s_\tau)$ .
    Update  $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_{\tau=t}^{t+H} \frac{1}{2} \|v_\psi(s_\tau) - V_\lambda(s_\tau)\|^2$ .

  // Environment interaction
   $o_1 \leftarrow \text{env.reset}()$ 
  for time step  $t = 1..T$  do
    Compute  $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$  from history.
    Compute  $a_t \sim q_\phi(a_t | s_t)$  with the action model.
    Add exploration noise to action.
     $r_t, o_{t+1} \leftarrow \text{env.step}(a_t)$ .
    Add experience to dataset  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$ .
  
```

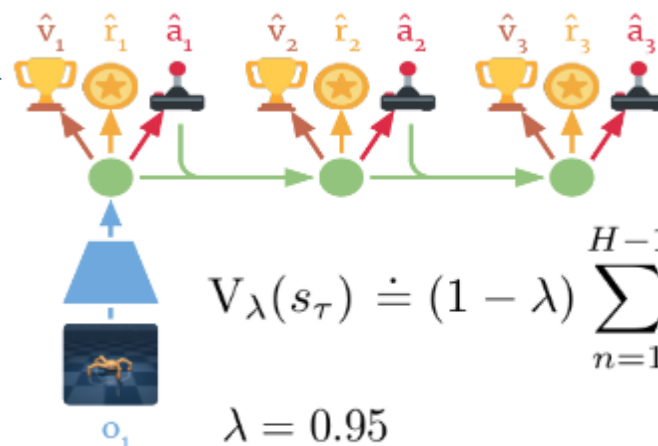
Unlike the original Dreamer algorithm, our approach does **not include online data collection or online world model learning**. For computational efficiency, we rely on a fixed pre-trained world model and train the behavior policy on latent states obtained from trajectories generated by a **random policy**. As a consequence, the policy is optimized under a fixed data distribution rather than an evolving replay buffer

Me:

Training step = 10^4
 Initial latent states = 64/32
 Horizon imagination = 15/50
 Imagine transition for step = 960/1600

Paper:

$T = 1000$
 $C = 100$
 Environment step = 10^6
 Training step = 10^5
 Initial latent states = 2500
 Horizon imagination = 10
 Imagine transition for step = 25000



$$V_\lambda(s_\tau) \doteq (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} V_N^n(s_\tau) + \lambda^{H-1} V_N^H(s_\tau)$$

$$\lambda = 0.95$$

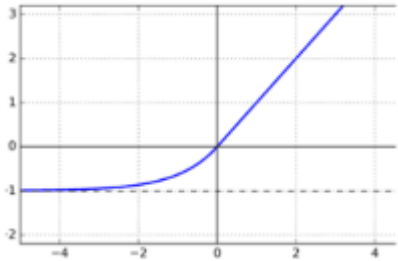
$$\gamma = 0.99$$

$$V_N^k(s_\tau) \doteq E_{q_\theta, q_\phi} \left(\sum_{n=\tau}^{h-1} \gamma^{n-\tau} r_n + \gamma^{h-\tau} v_\psi(s_h) \right)$$

Learning objectives:

Neural network:

- Features dim = 288
- 3 dense layer
- Size 256
- ELU activations



$$q_{\phi}(a_{\tau} \mid s_{\tau})$$

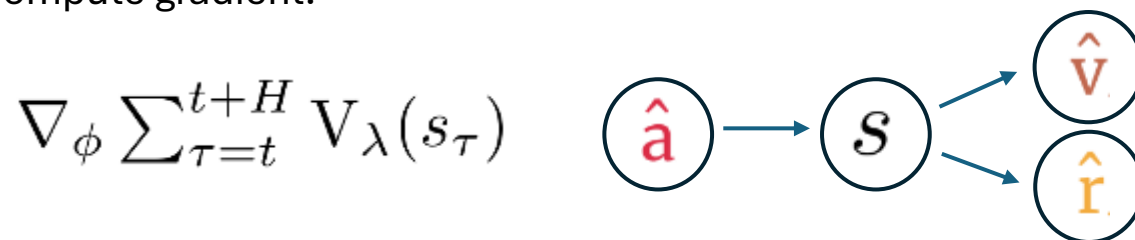
- Sample actions from **three Gaussian** with **Reparametrisation trick** to hold differentiability
- **Tanh-transformed** output

$$a_{\tau} = \tanh(\mu_{\phi}(s_{\tau}) + \sigma_{\phi}(s_{\tau}) \epsilon), \quad \epsilon \sim \text{Normal}(0, \mathbb{I}).$$
- Scaling and traslation to the right action intervals
- $$\max_{\phi} \mathbb{E}_{q_{\theta}, q_{\phi}} \left(\sum_{\tau=t}^{t+H} V_{\lambda}(s_{\tau}) \right)$$

$$v_{\psi}(s_{\tau})$$

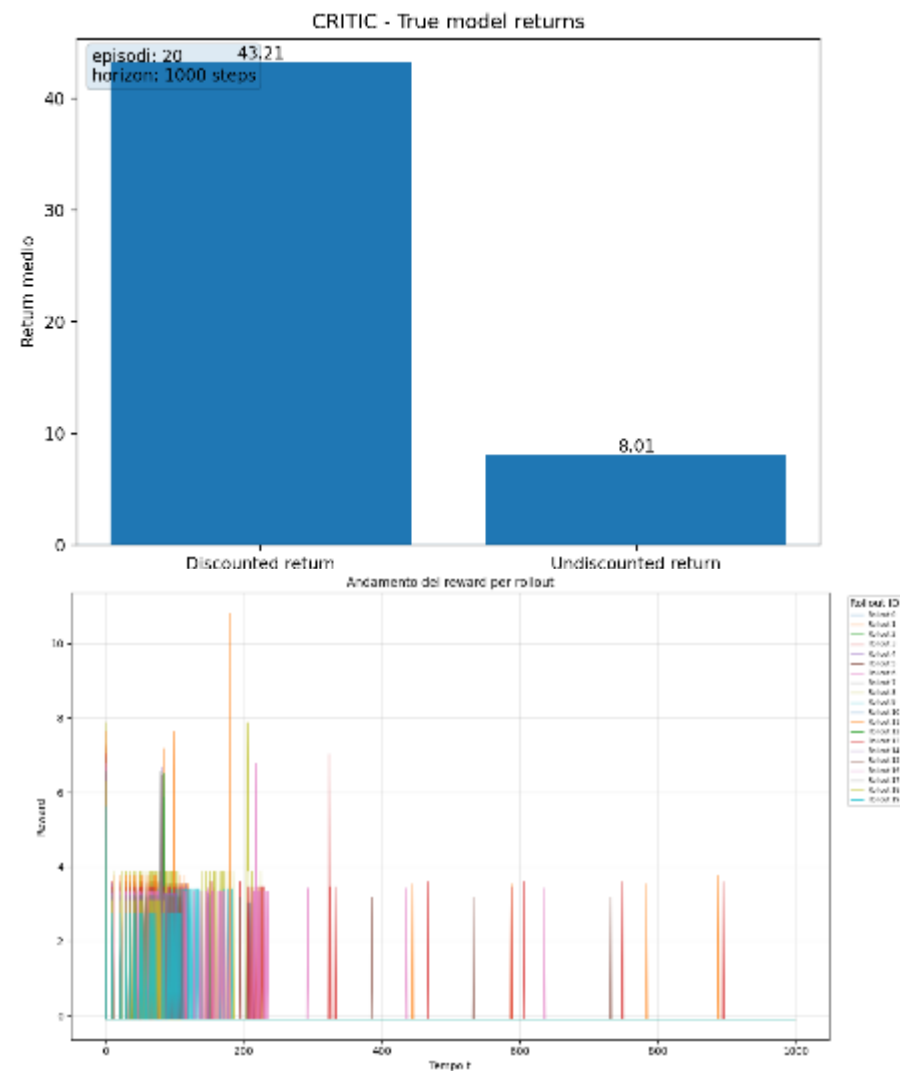
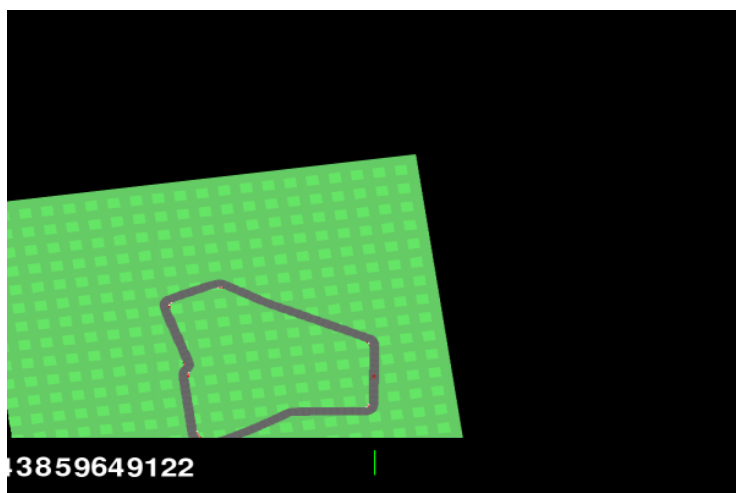
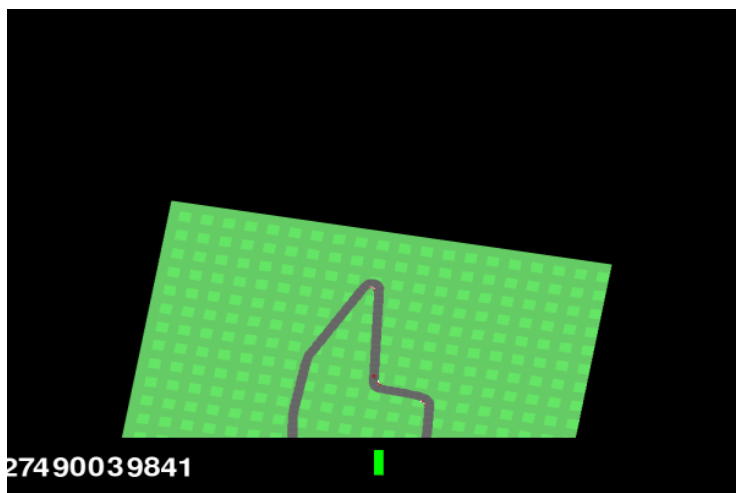
- Simple **real valued** parametrized function
- $$\min_{\psi} \mathbb{E}_{q_{\theta}, q_{\phi}} \left(\sum_{\tau=t}^{t+H} \frac{1}{2} \|v_{\psi}(s_{\tau}) - V_{\lambda}(s_{\tau})\|^2 \right)$$

Analytically compute gradient:

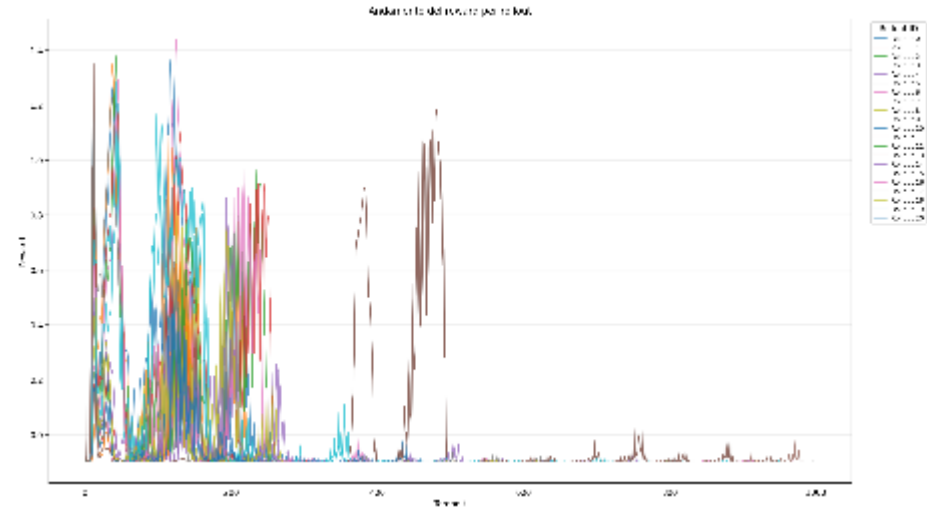
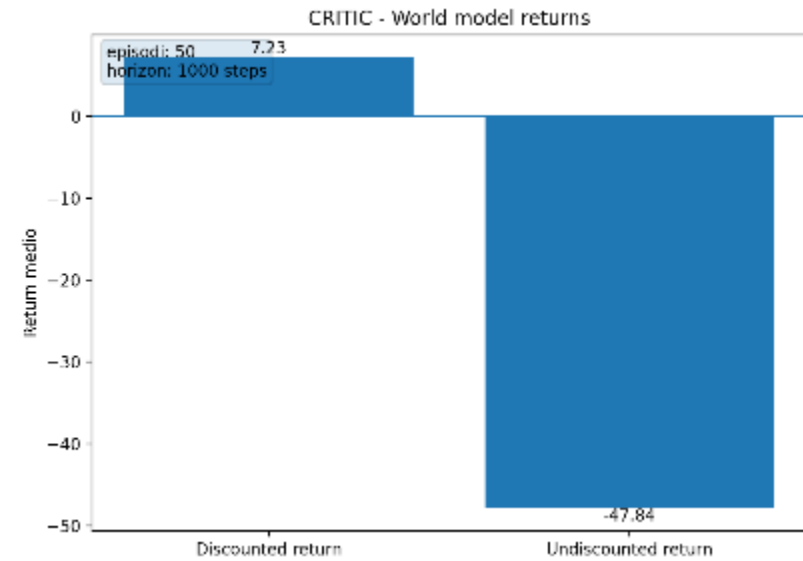
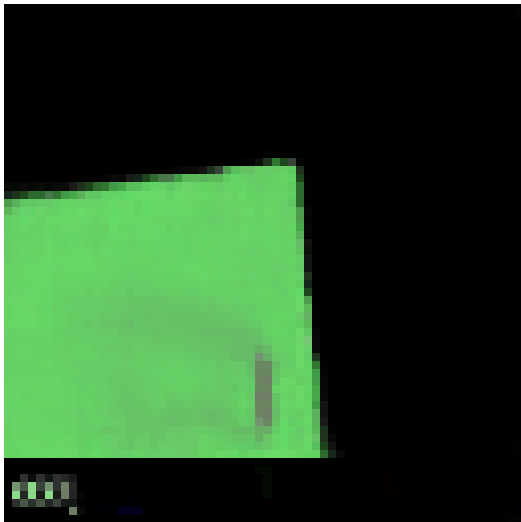
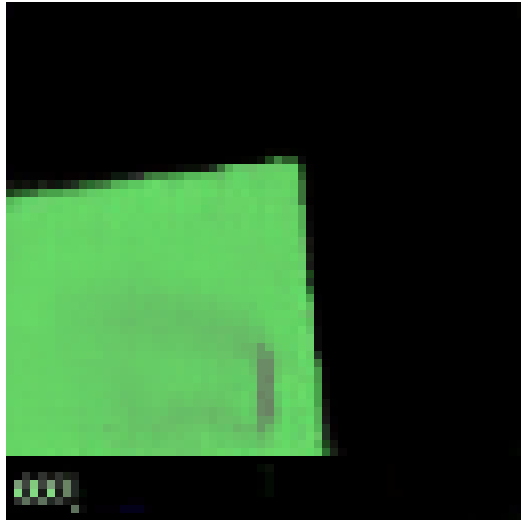


Dreamer stands out because it uses a **differentiable world model** to generate latent rollouts and propagate the gradients of rewards and values through the imagined transitions back to the actor parameters

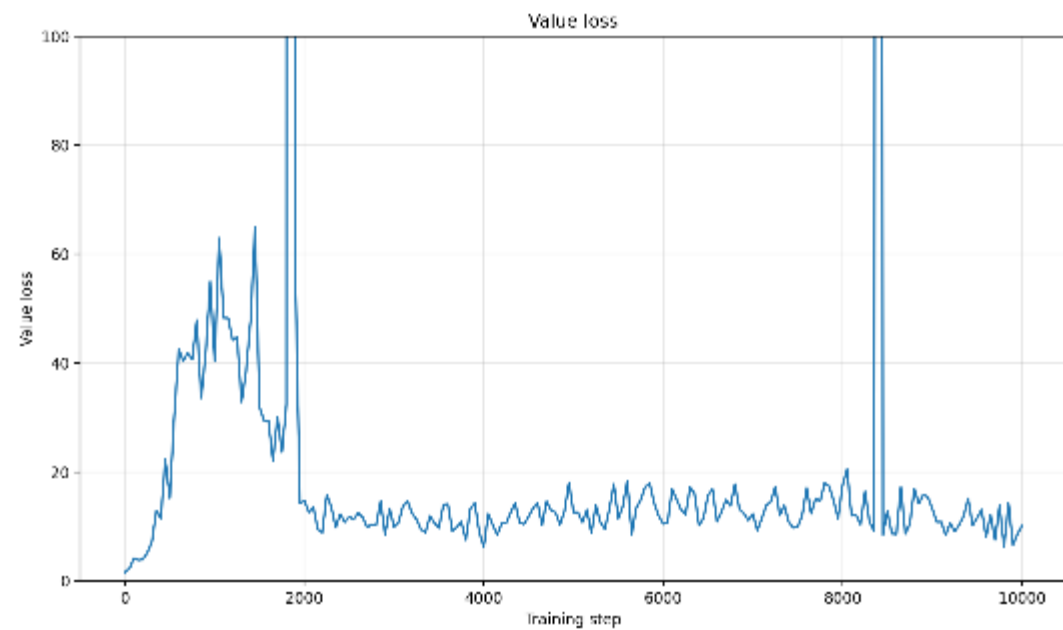
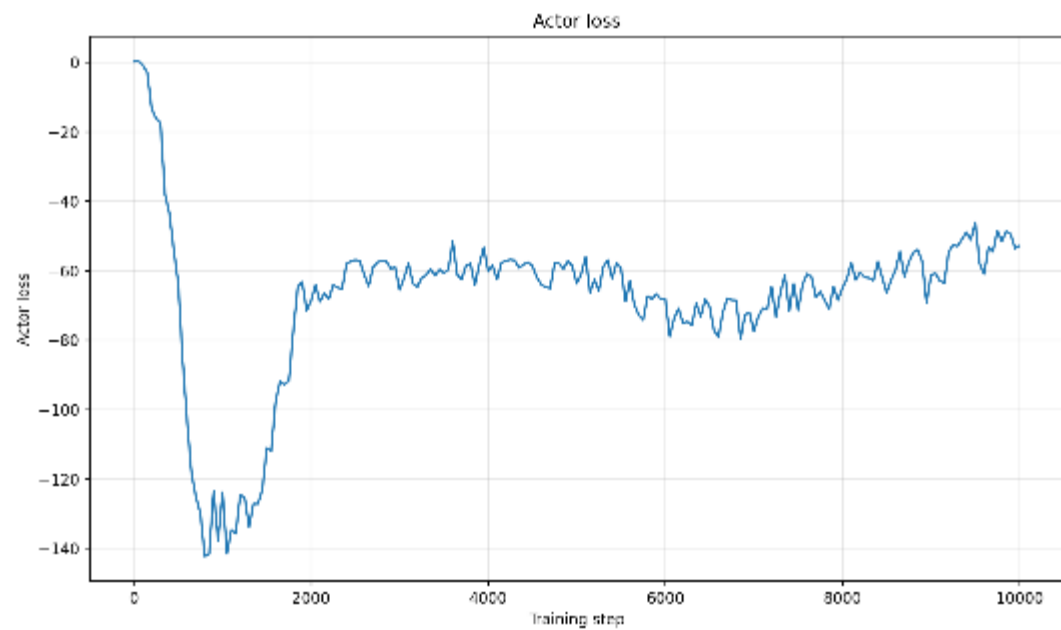
Results



Learn something in world model?



Training



Train analysis:



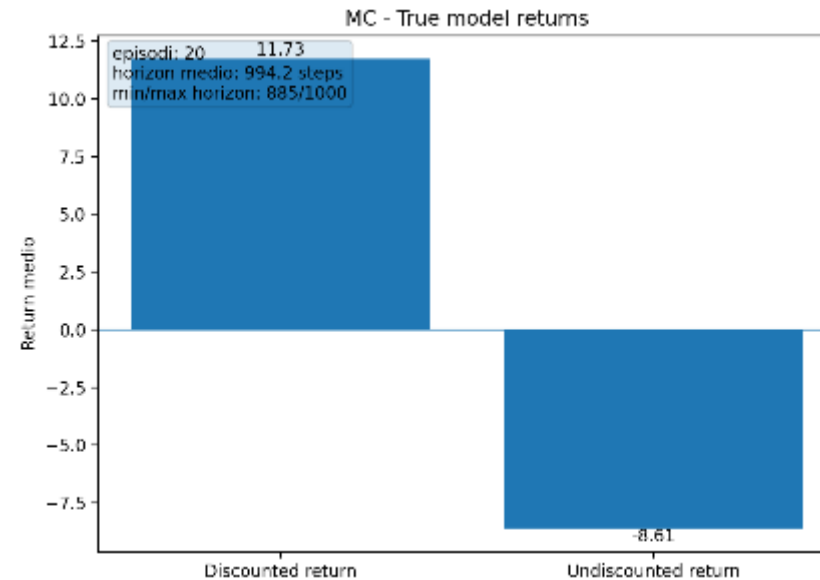
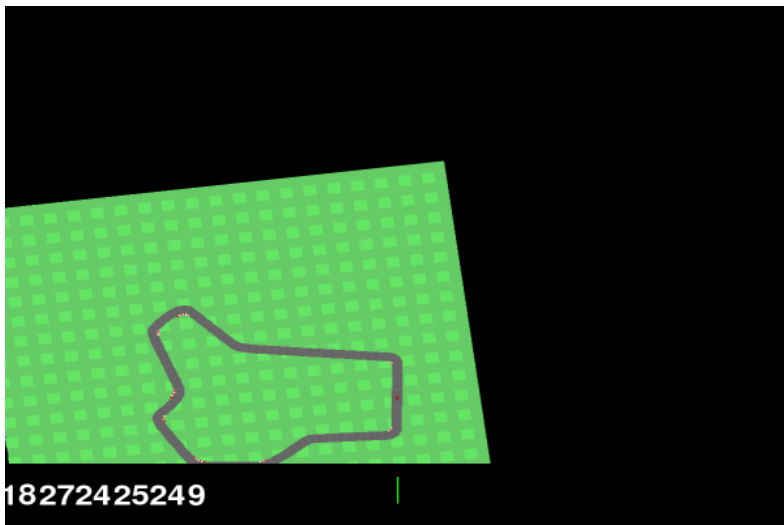
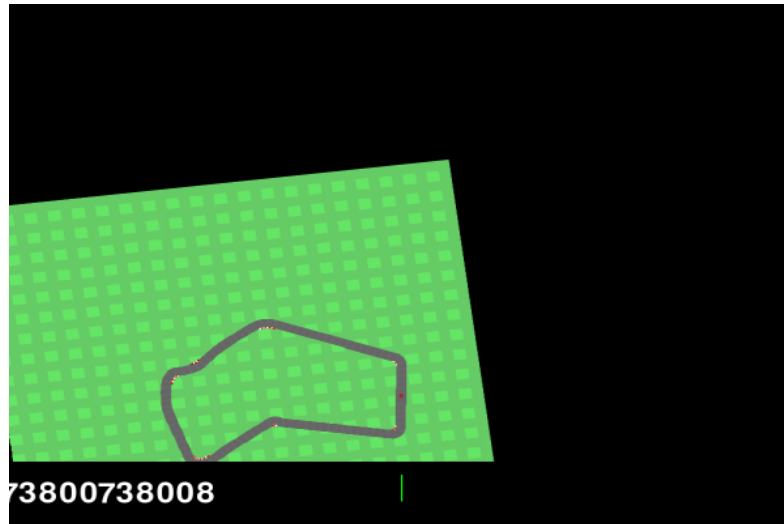
All the contribute came from the value model !
The actor learn to maximize the value and not the reward

$$\text{mean_reward} = \frac{1}{HB} \sum_{t=0}^{H-1} \sum_{b=1}^B \hat{r}_{t,b}$$

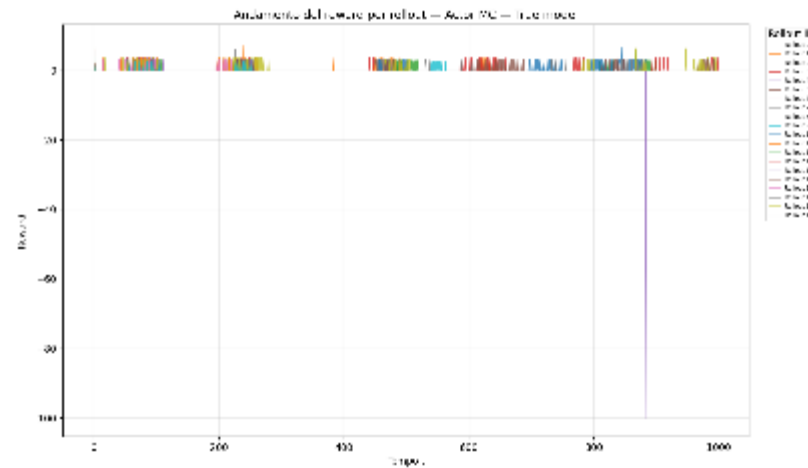
$$\text{mean_value} = \frac{1}{HB} \sum_{t=0}^{H-1} \sum_{b=1}^B V_{\psi}(s_{t,b})$$

$$\text{mean_return} = \frac{1}{HB} \sum_{t=0}^{H-1} \sum_{b=1}^B G_{t,b}^{\lambda}$$

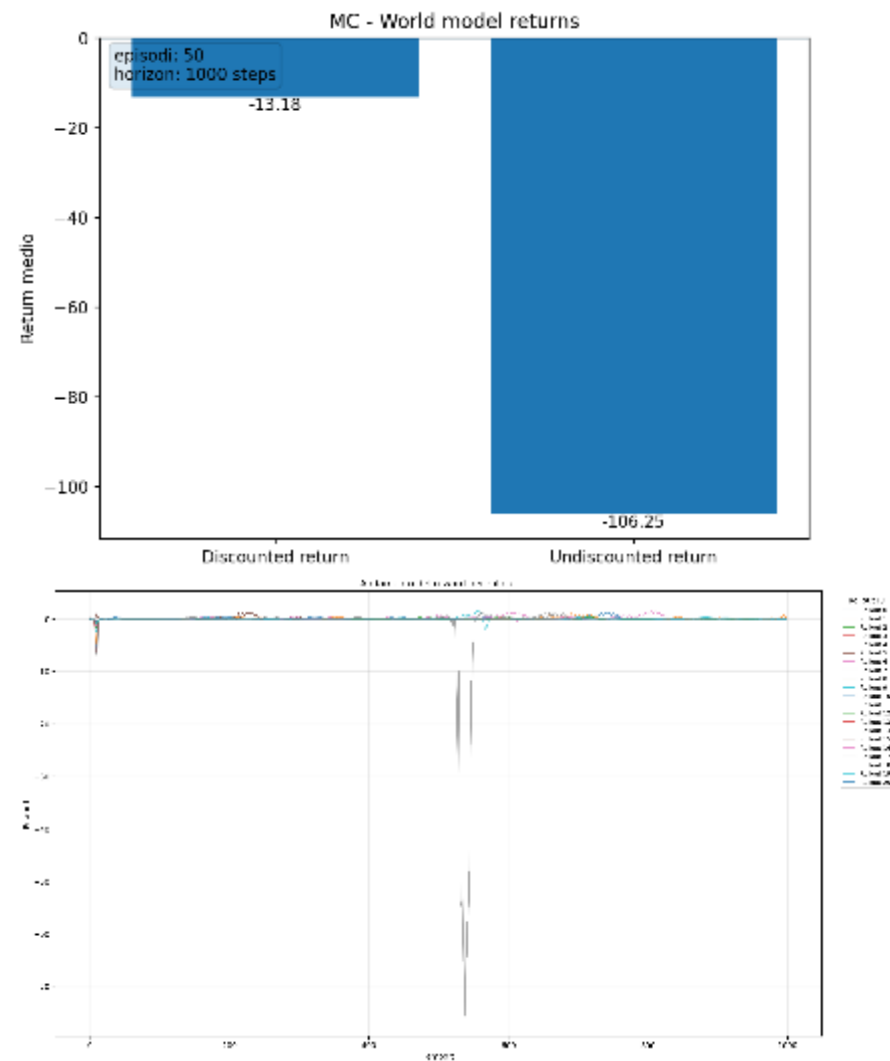
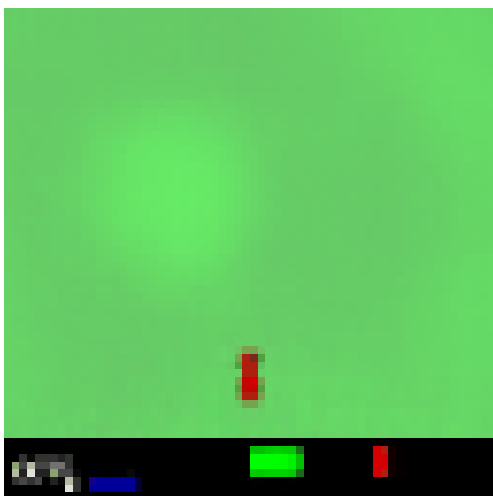
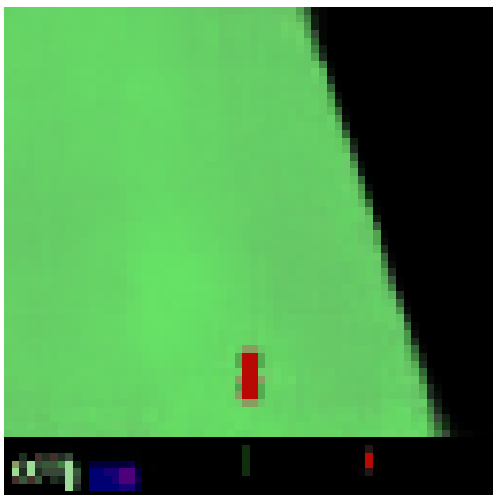
Maybe a Monte Carlo approach...



Lambda-returns already include a Monte Carlo component, so this behavior is not unexpected

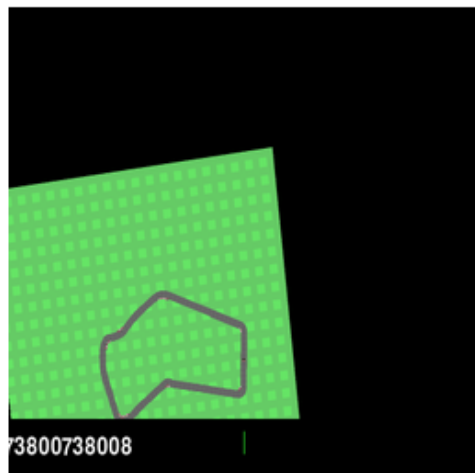


Monte Carlo World model

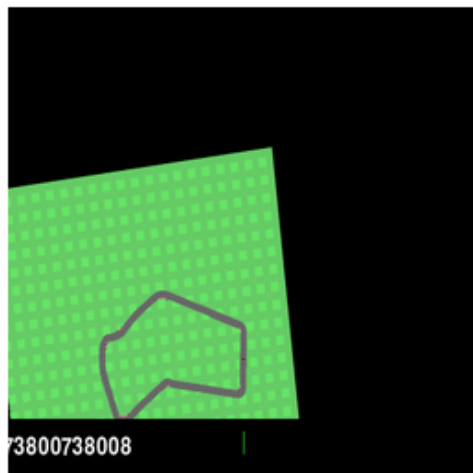


Compare with good controller

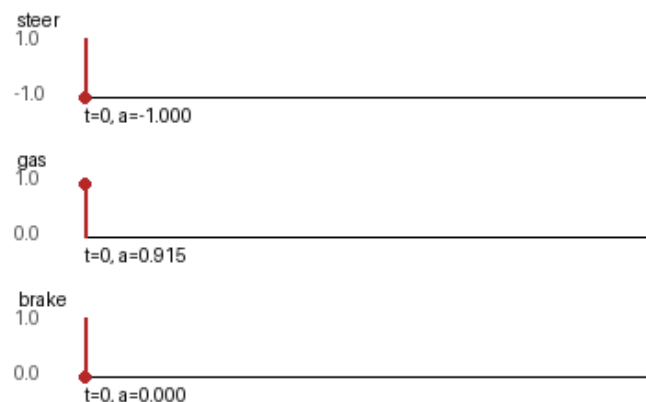
Dreamer real env | t=0 | return=7.01



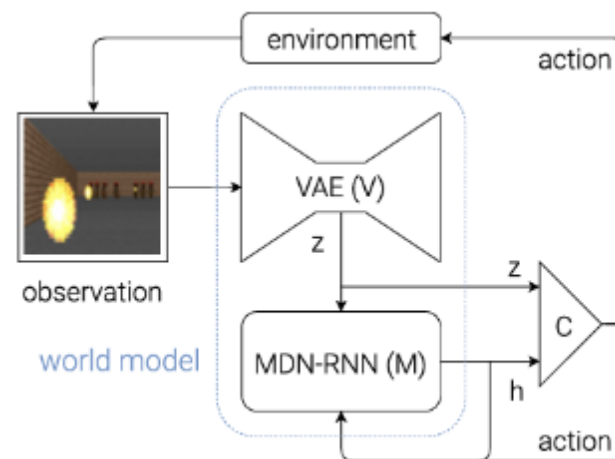
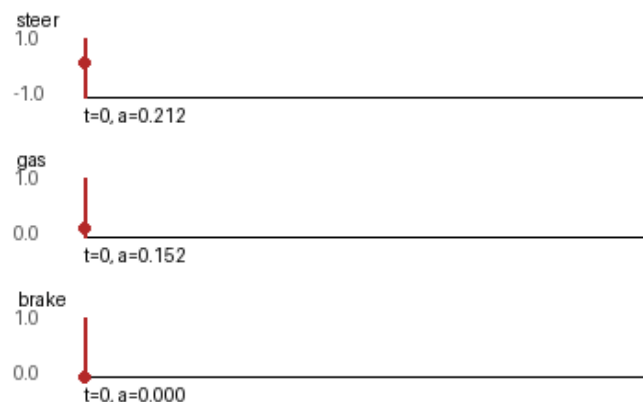
Evo real env | t=0 | return=917.70



Dreamer actions

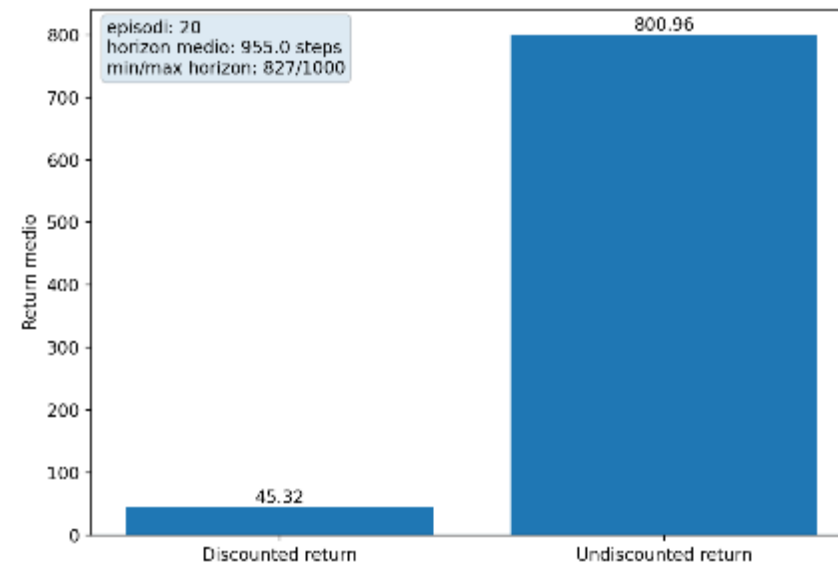


Evoactions



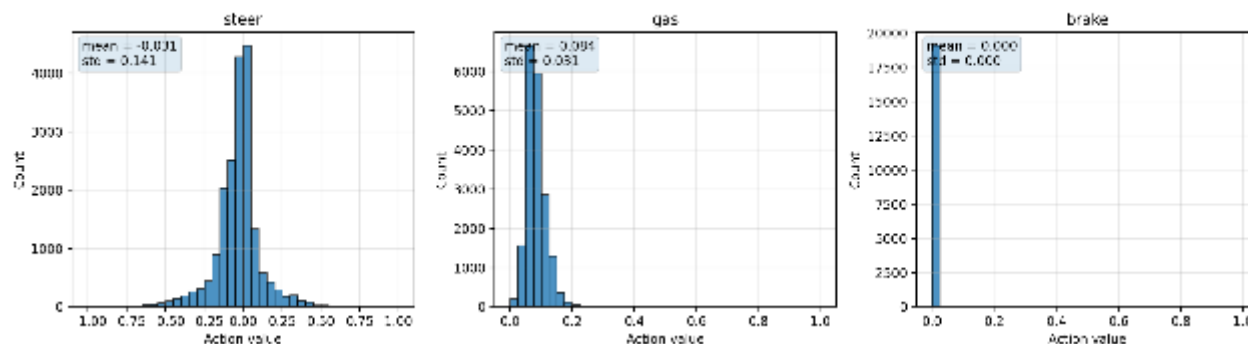
To optimize the parameters of C is used **CMA-ES** (evolution algorithm) since it is known to work well for solution spaces of up to a few thousand parameters (C have 867 parameter)

EVO - True model returns

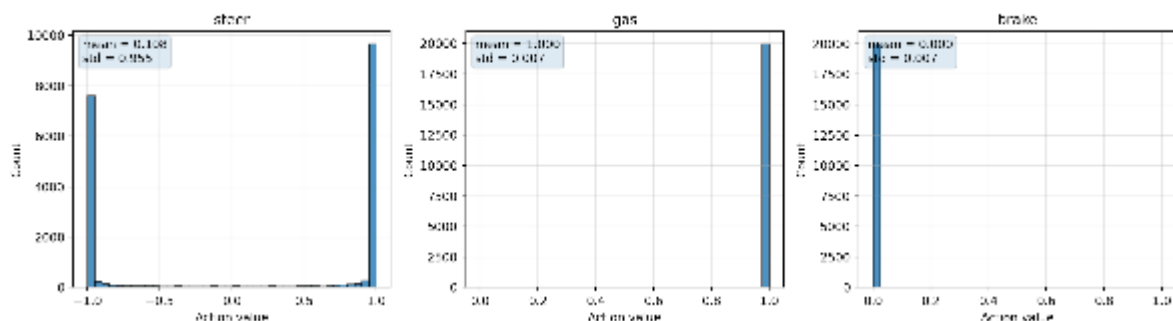


Action distribution comparison

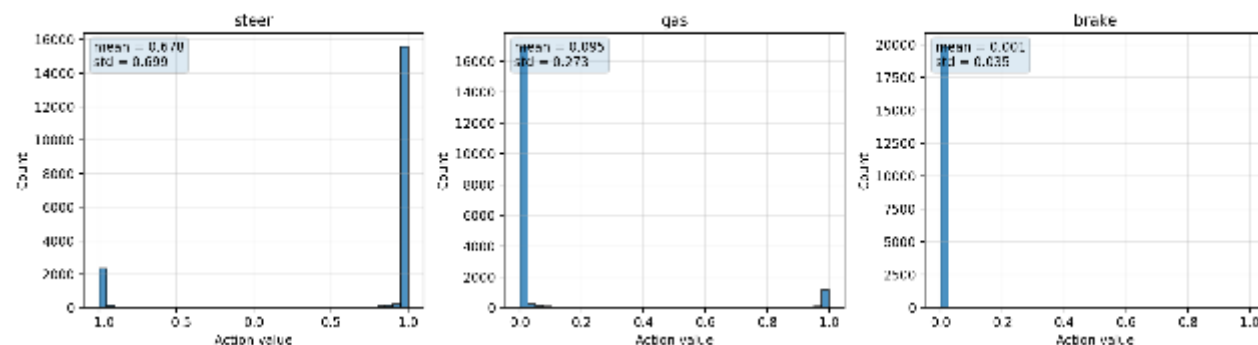
EVD - True model action distributions
rollouts = 20 | total steps = 19100



Dreamer CRITIC - True model action distributions
rollouts = 20 | total steps = 20000



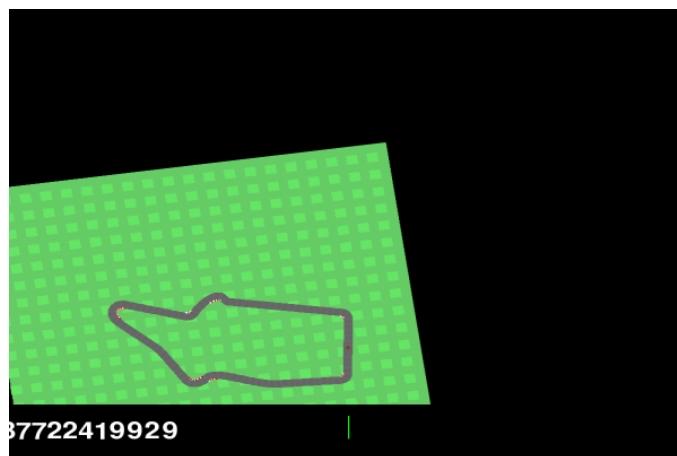
Dreamer MC - True model action distributions
rollouts = 20 | total steps = 19885



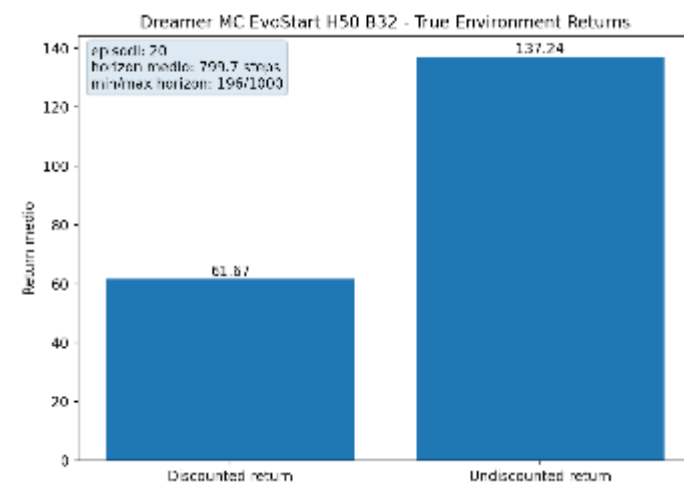
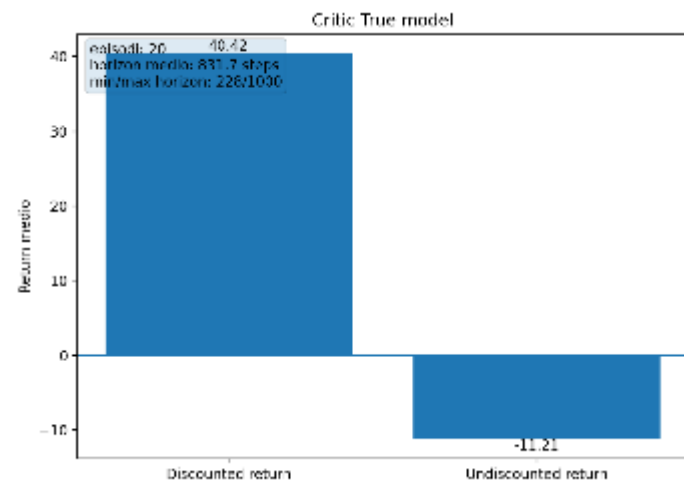
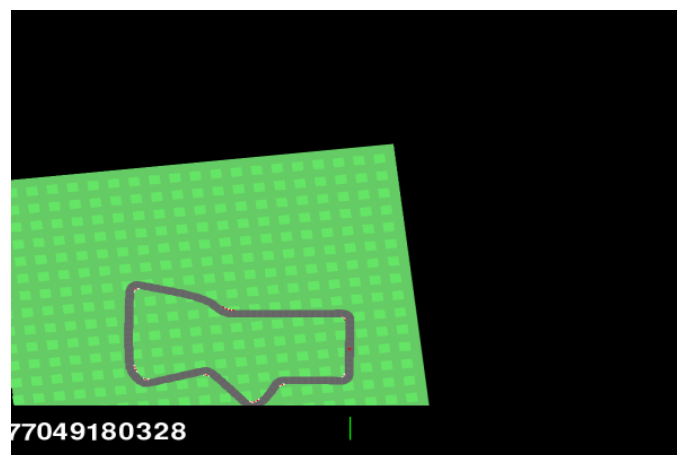
Longer horizon

Horizon 50 Initial latent state from Evo Policy

CRITIC

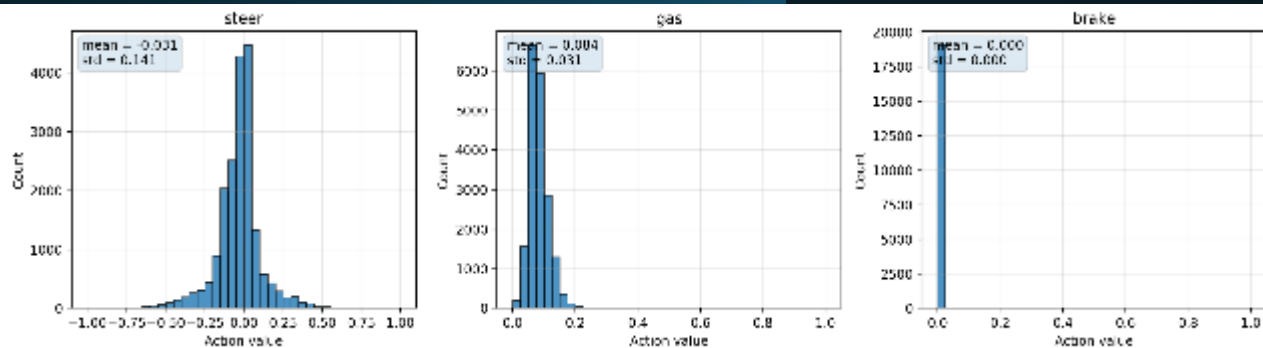


MONTÉ CARLO

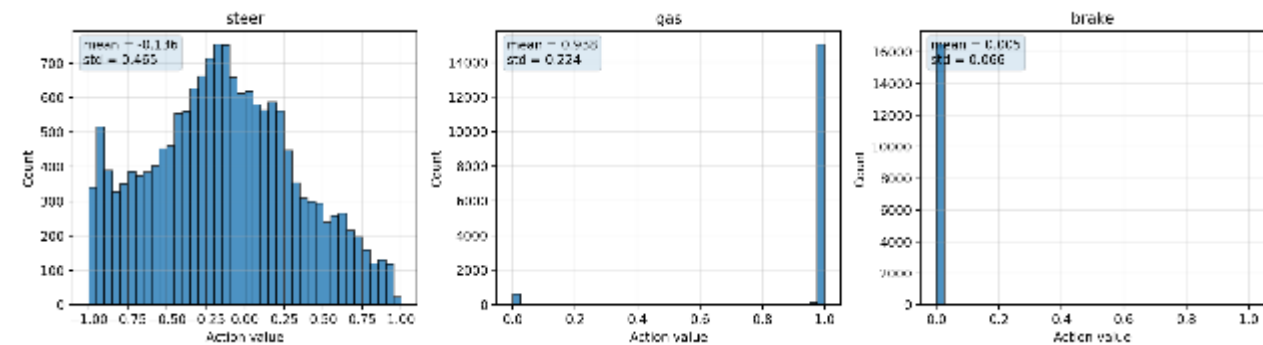


Actions distribution long horizon

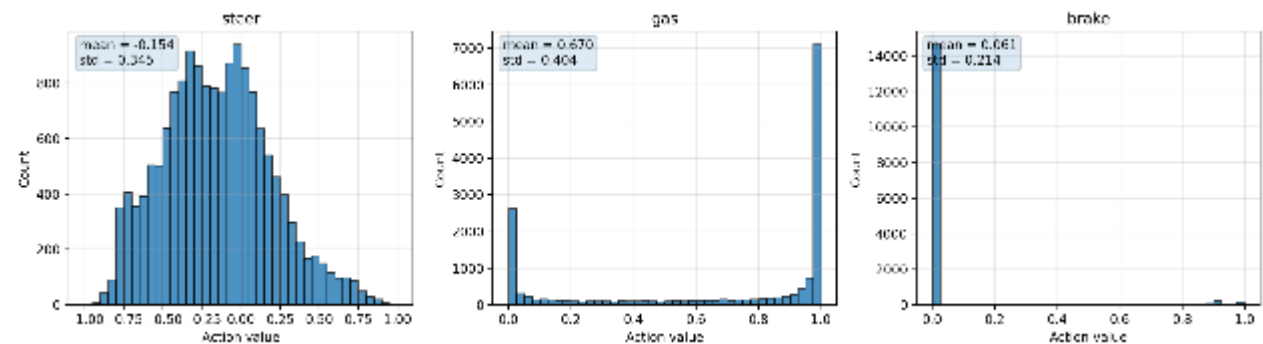
EVOLUTIONARY



CRITIC

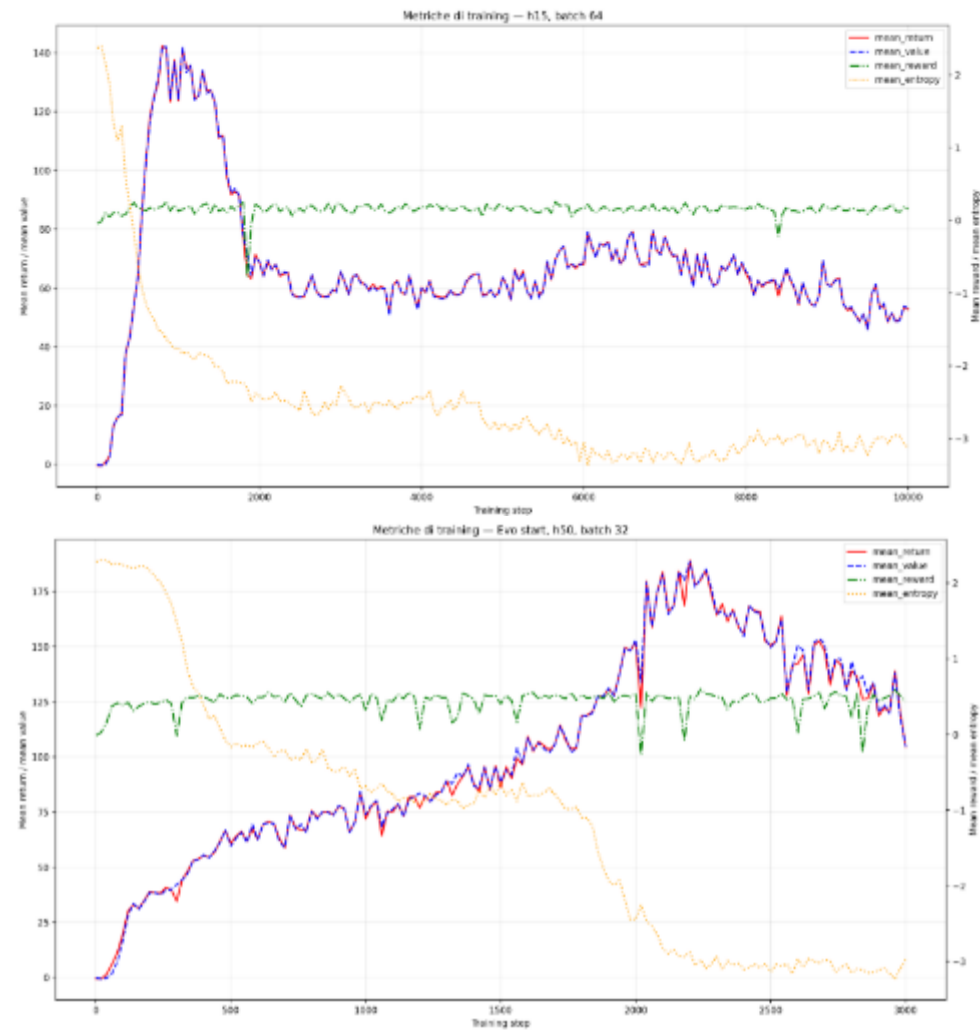
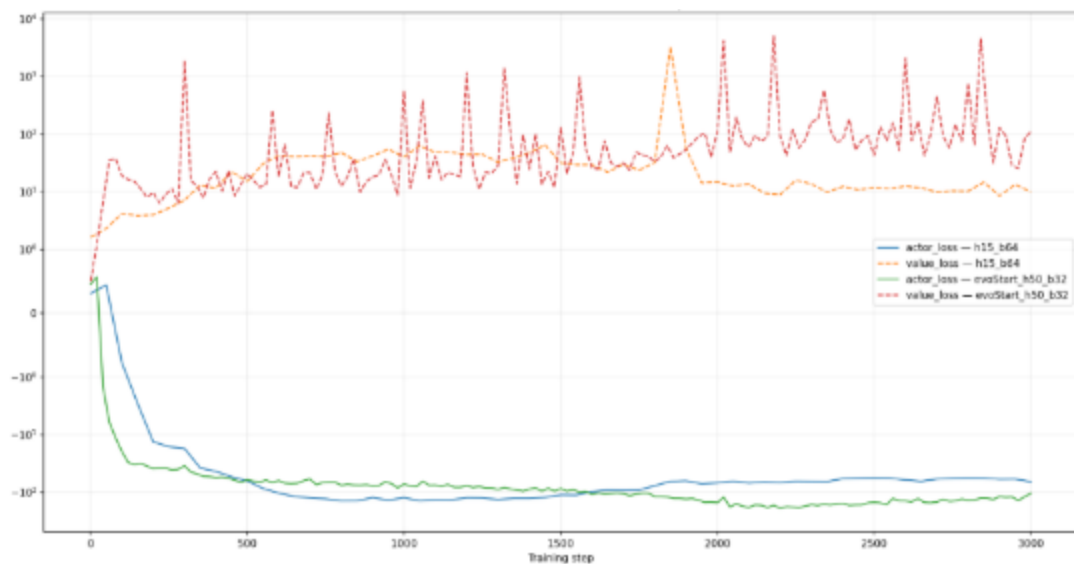


MONTE CARLO



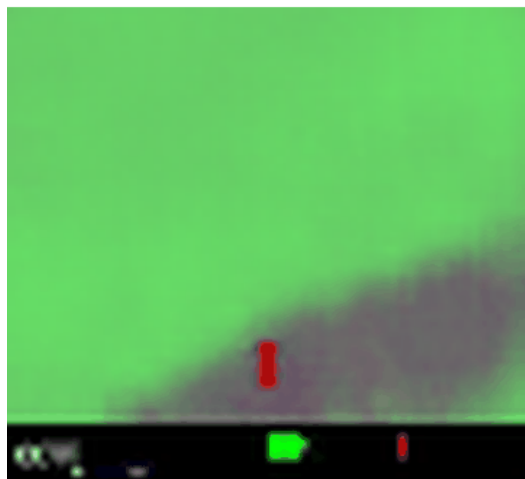
Mean on 20 episodes

Train analysis long horizon

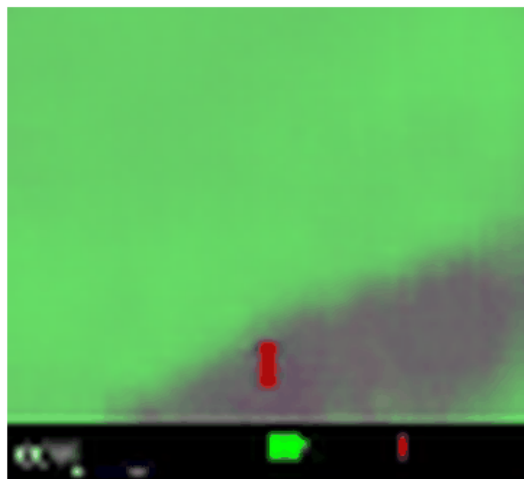


But in the world model...

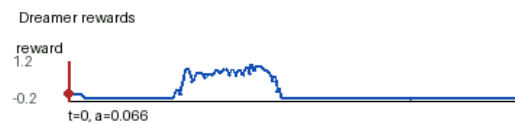
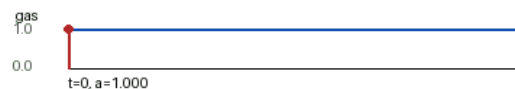
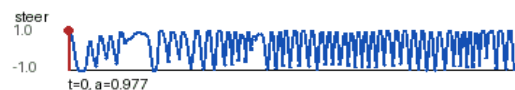
Dreamer world model rollout | frame 0



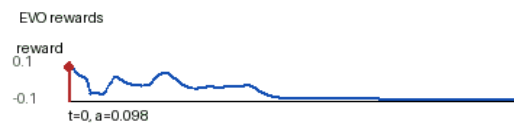
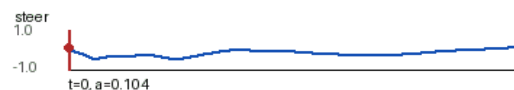
EVO world model rollout | frame 0



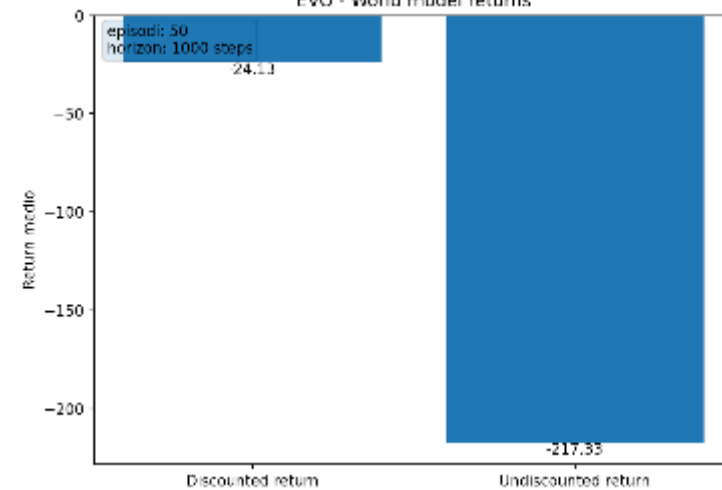
Dreamer actions



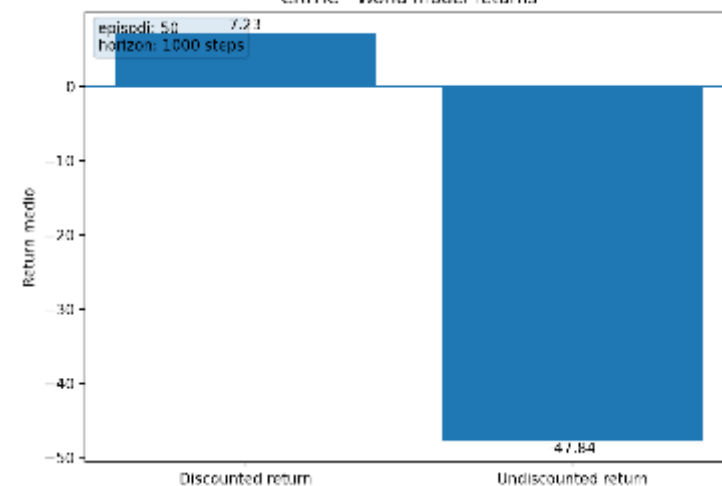
EVO actions



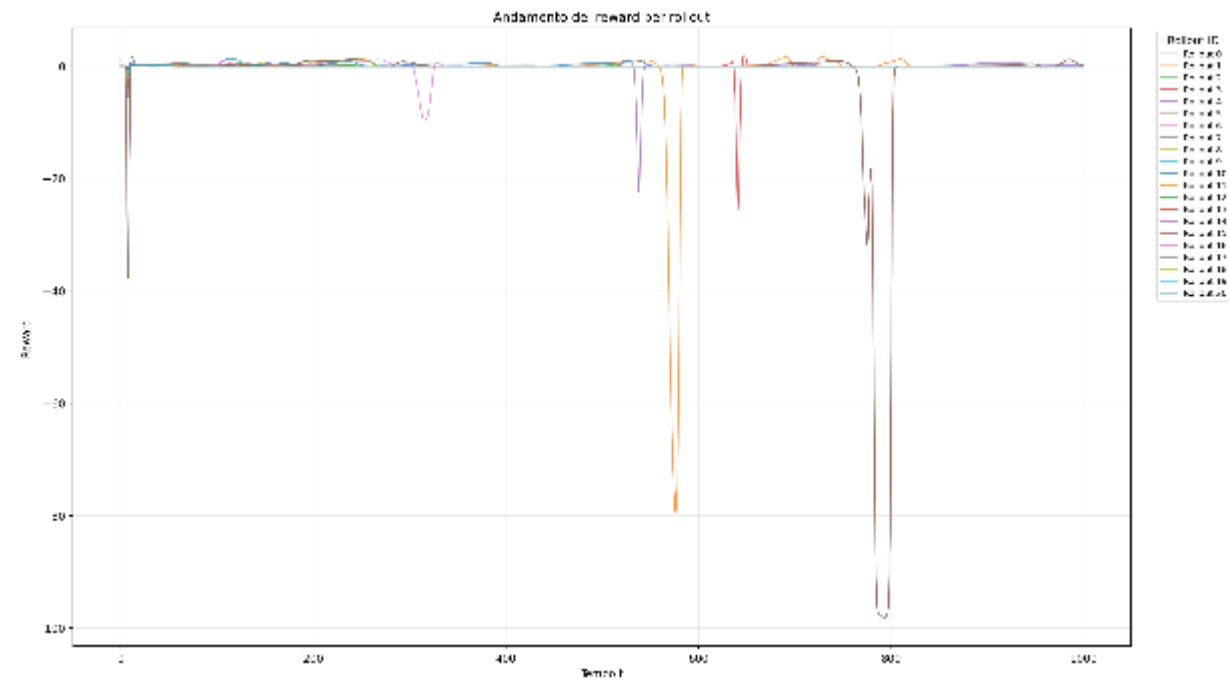
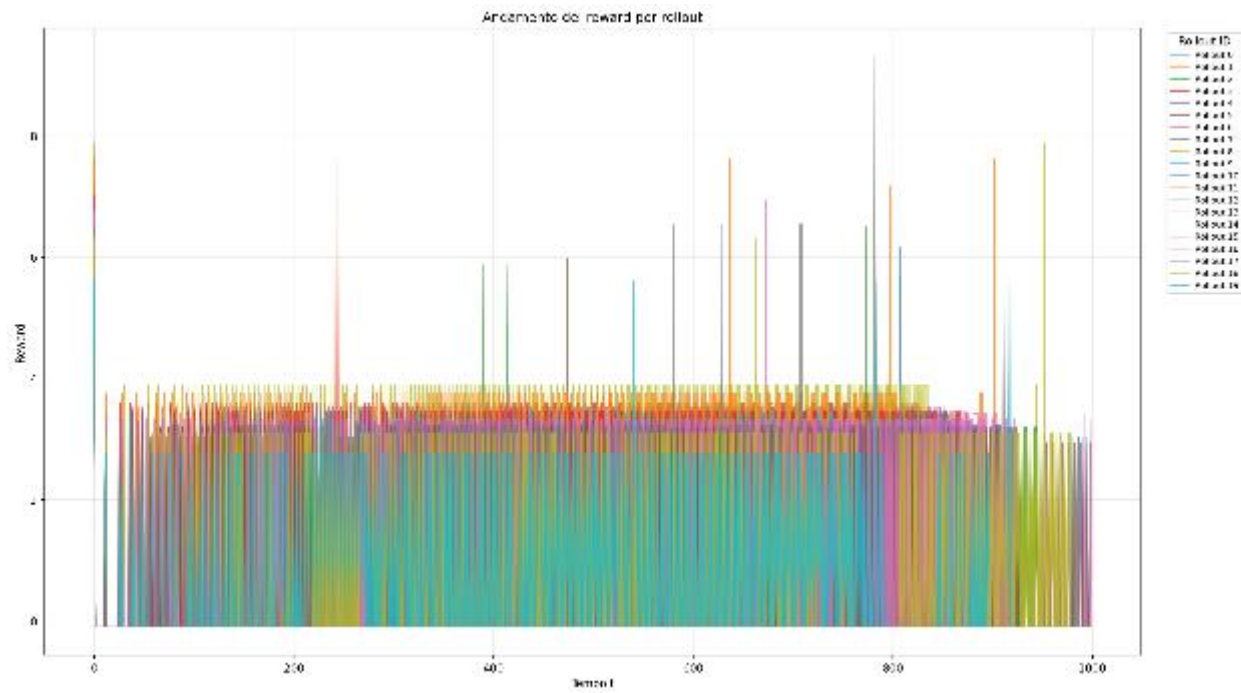
EVO - World model returns



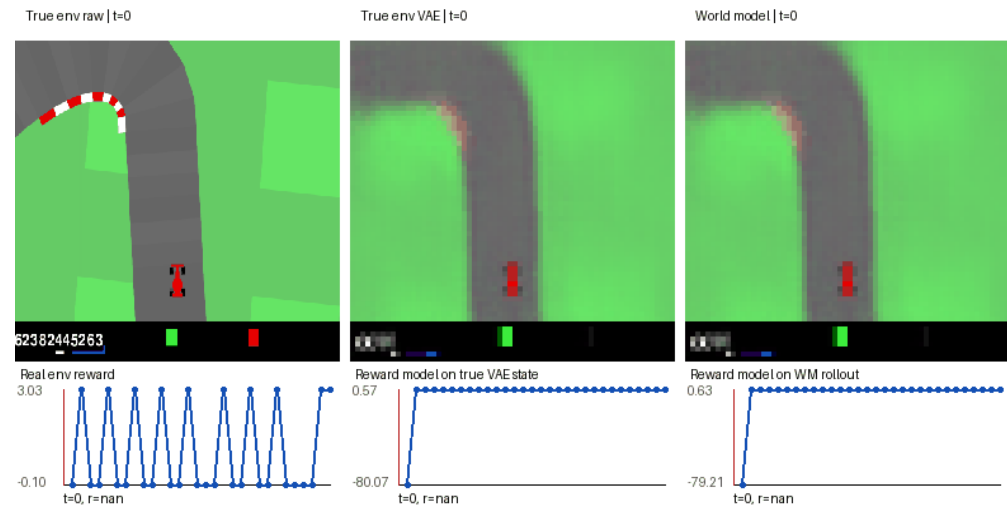
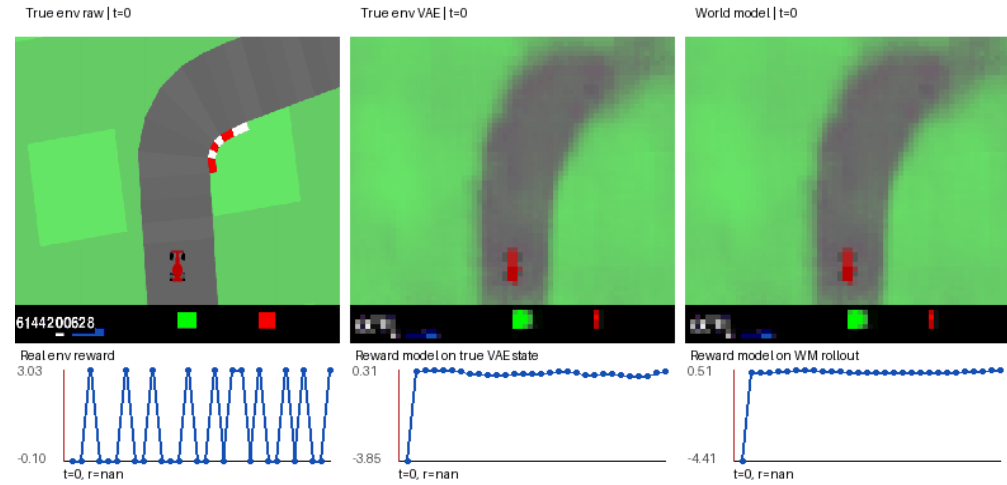
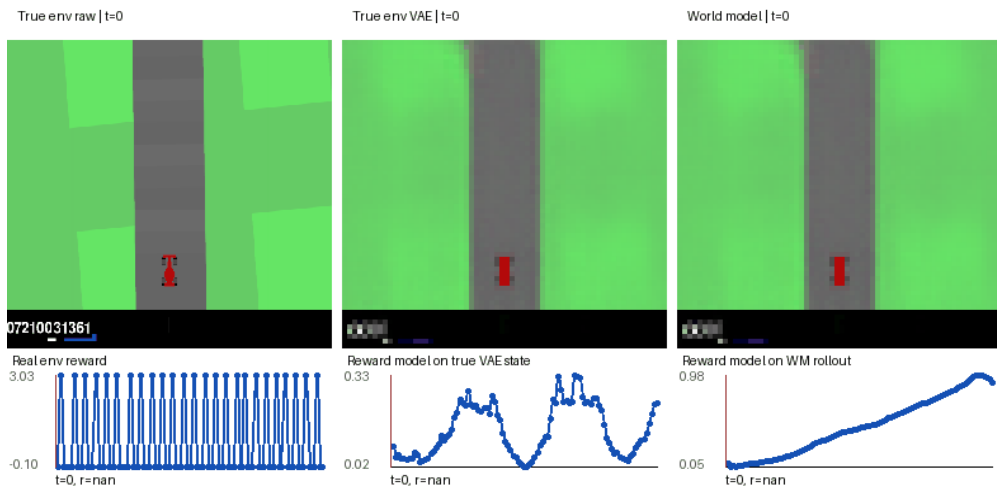
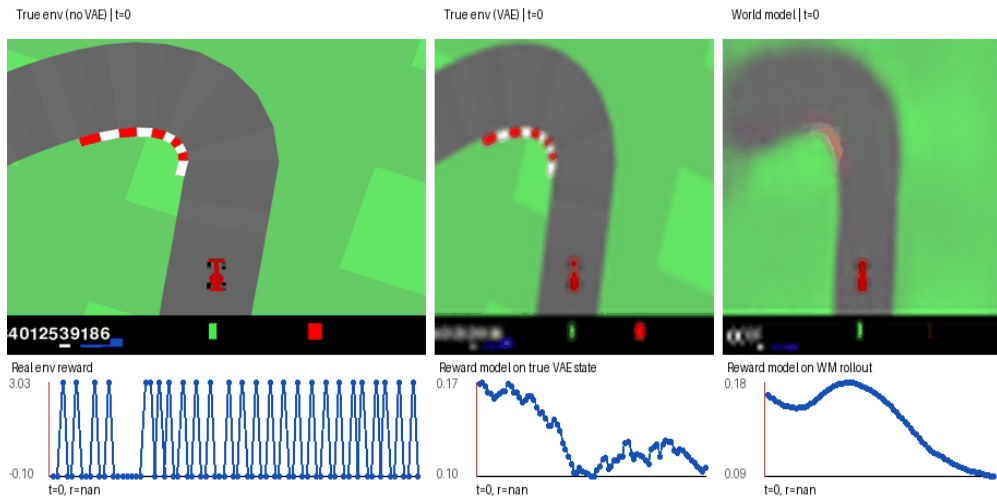
CRITIC - World model returns



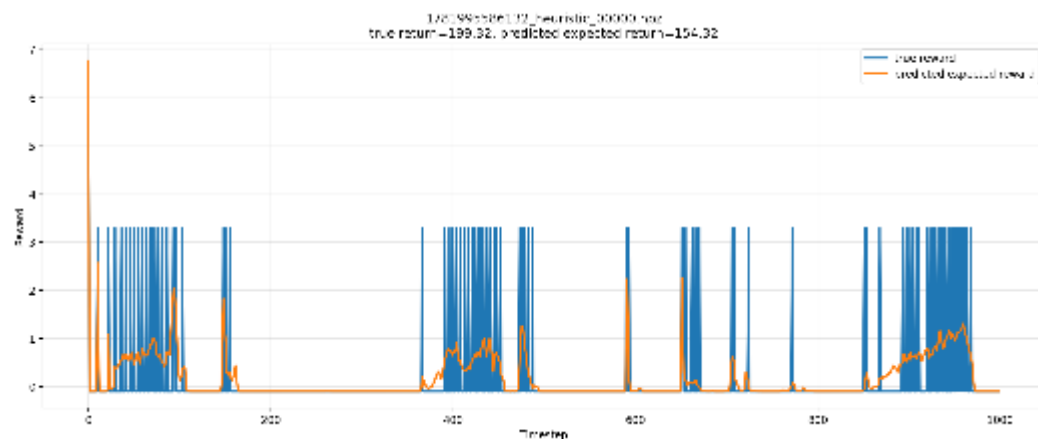
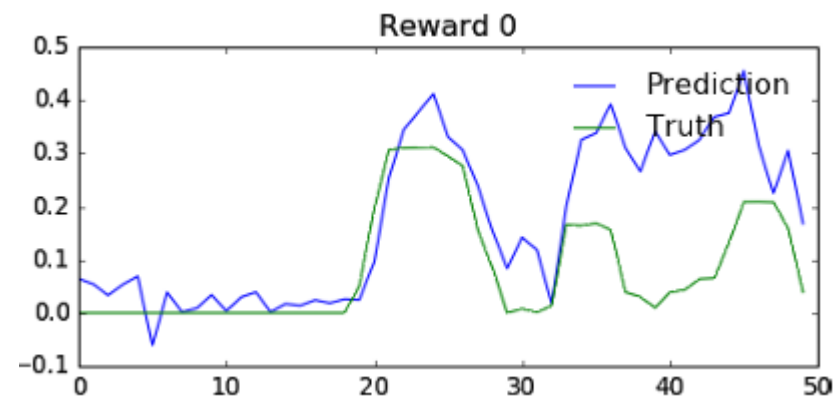
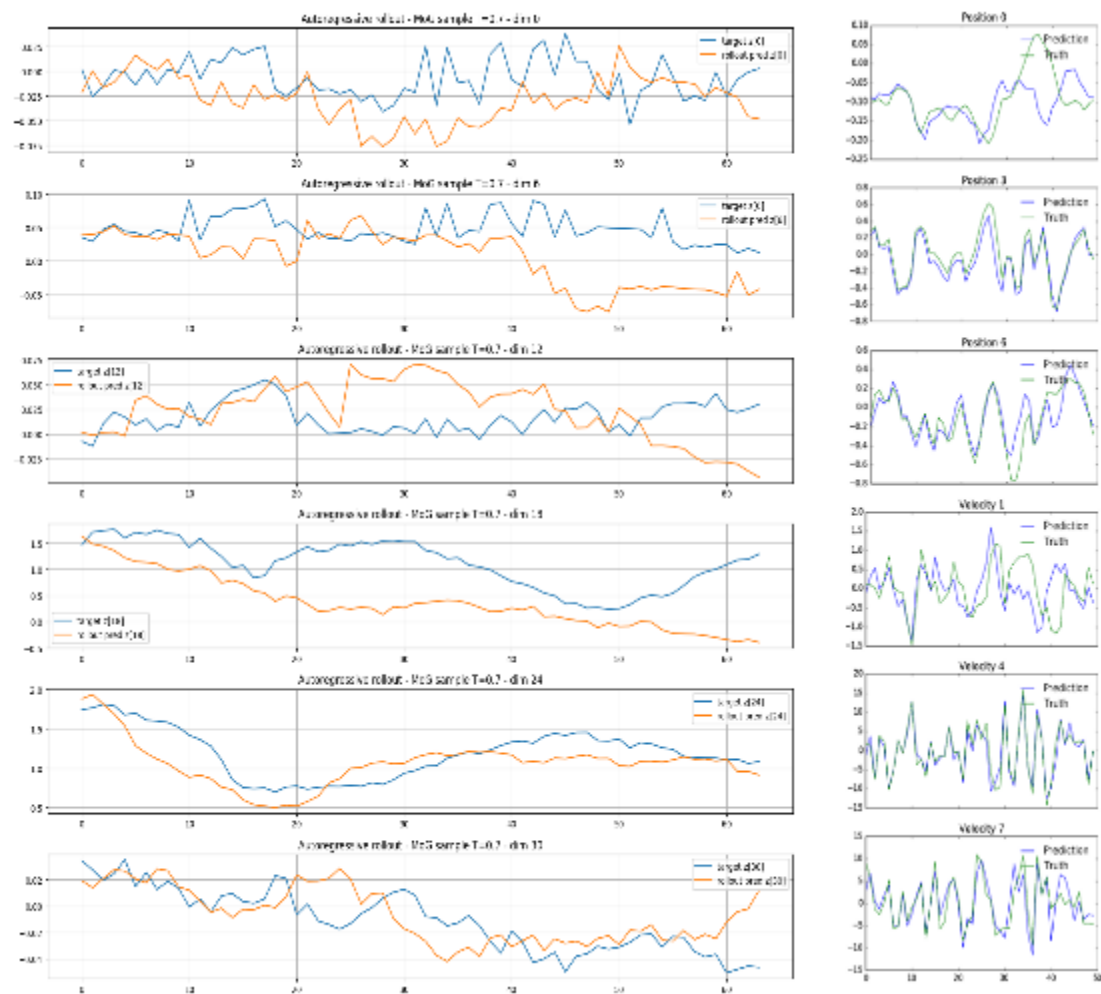
Evo in world model



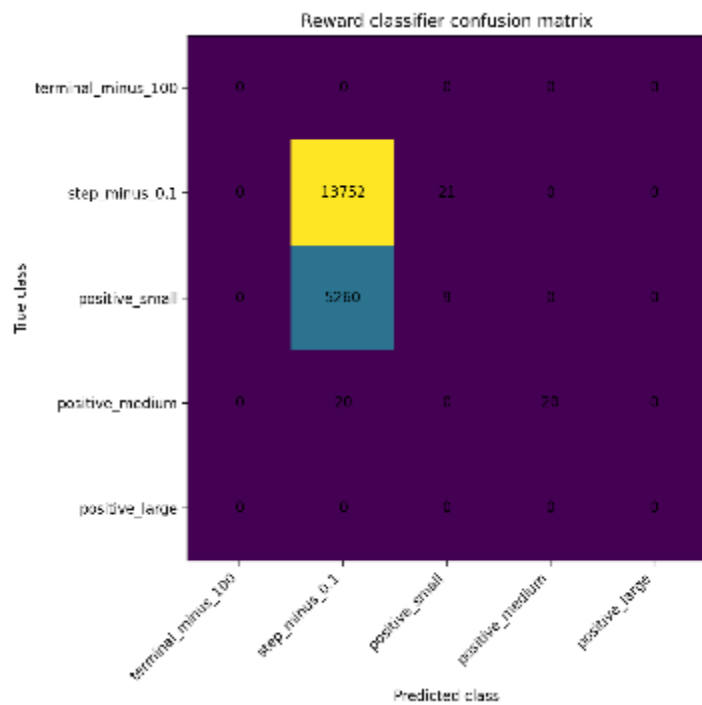
World model quality



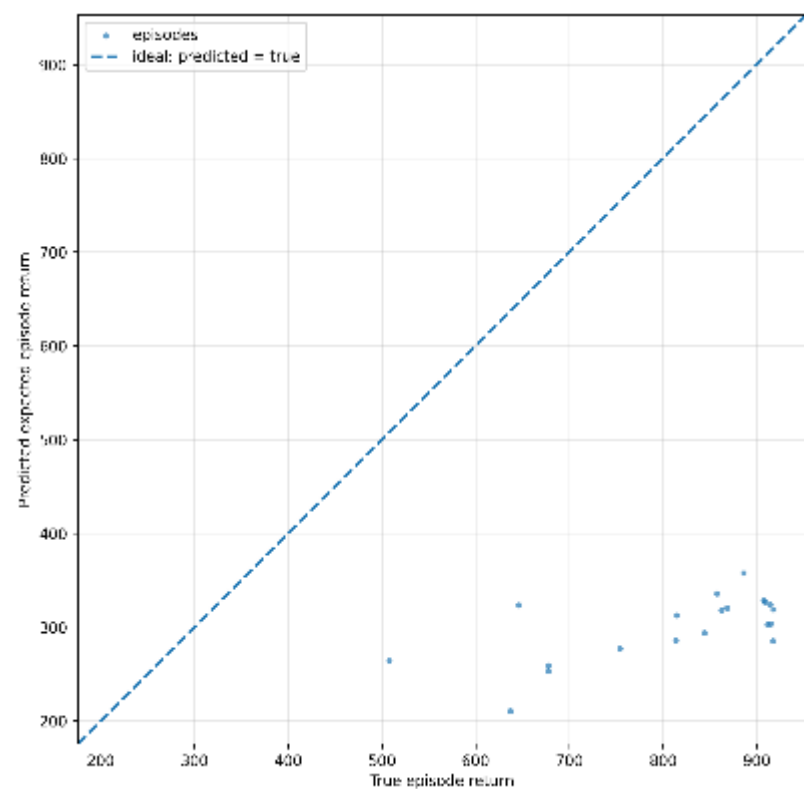
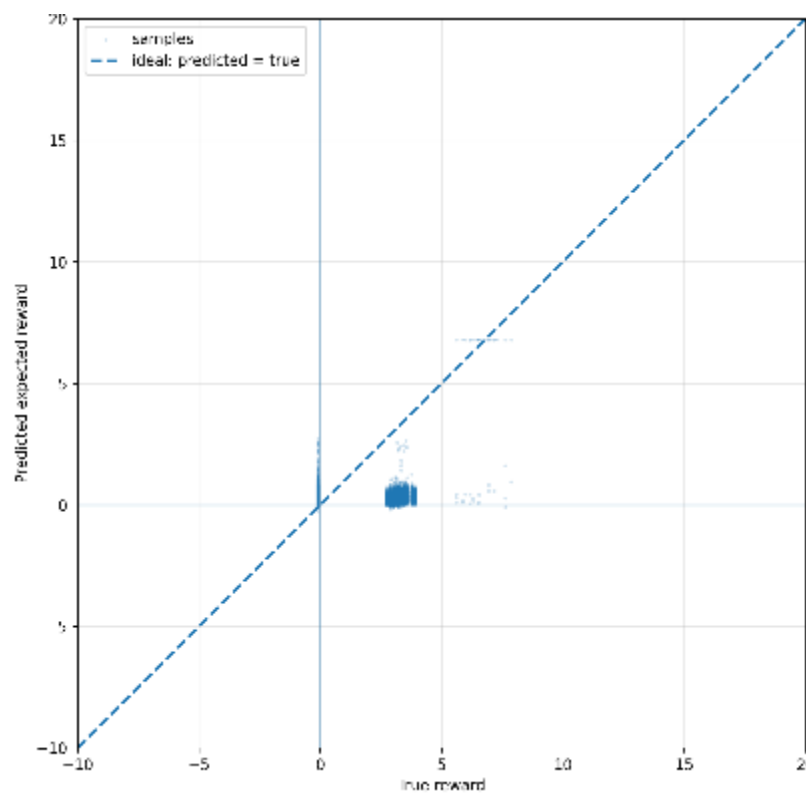
Comparison with PlaNet World model



Reward model



$[-100.0, -0.1, 3.3, 6.8, 11.0]$



Limitations of Policy Learning

- The policy was trained without **online data collection**.
- The **world model was kept frozen** for computational reasons: jointly retraining the VAE, recurrent dynamics model, reward model, actor, and value model would require significantly larger computational resources. The VAE and RNN were **trained separately**, following the practical approach of the original World Models paper.
- Unlike Dreamer, our RNN is mainly **trained for one-step latent prediction**, while Dreamer optimizes a world model objective that supports multi-step latent predictions and jointly learns task-relevant representations.
- Random-policy data can be sufficient to learn a reasonable latent dynamics model for CarRacing, but it is less suitable for learning a **task-relevant reward model**. A good reward model requires data from states actually visited by improving policies, especially states related to track following, progress, and failure modes.

.

References:

- [1] D. Ha and J. Schmidhuber, World Models, arXiv:1803.10122, 2018.
- [2] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, Learning Latent Dynamics for Planning from Pixels, Proceedings of the 36th International Conference on Machine Learning, PMLR, 2019.
- [3] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, Dream to Control: Learning Behaviors by Latent Imagination, International Conference on Learning Representations, 2020.
- [4] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.

Thanks for attention

